

Reviewer Pack — Sandpile-Group Order of Certificate Conflict Graph Lower-Bou...

Entry ef886a7094f0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 11:19:17 UTC

Sandpile-Group Order of Certificate Conflict Graph Lower-Bounds Q^{dt}

Entry ID: ef886a7094f0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 11:19:17 UTC

1. Conjecture

Field A (mathematical branch): Abelian sandpile models / chip-firing on finite graphs (Björner-Lovász-Shor; Holroyd-Levine-Mészáros-Peres-Propp-Wilson); the sandpile (critical) group $K(G)$ computed as the cokernel of the reduced graph Laplacian, equivalently the spanning-tree count by Kirchhoff's matrix-tree theorem **Field B** (complexity object): Decision-tree depth $Q^{dt}(f)$ of small Boolean functions $f: \{0,1\}^k \rightarrow \{0,1\}$, transported via the Raz-McKenzie / Göös-Pitassi-Watson Indexing-gadget lifting theorem to deterministic two-party communication complexity $D^{cc}(f \circ \text{IND}_b)$

Statement:

For every Boolean $f: \{0,1\}^k \rightarrow \{0,1\}$, let G_f be the graph whose vertices are the minimal 1-certificates of f and whose edges $\{C, C'\}$ connect certificates that fix some common variable to opposite values (call these 'conflicts'). Then $\log_2 |K(G_f)| \leq Q^{dt}(f) \cdot \log_2(k+1)$, where $|K(G_f)|$ is the order of the abelian sandpile group of G_f , which equals the number of spanning trees of G_f by Kirchhoff (defined to be 1 if G_f has ≤ 1 vertex or is disconnected with each component a tree). By GPW lifting, this entails $D^{cc}(f \circ \text{IND}_b) \geq \Omega(\log_2 |K(G_f)| \cdot \log b / \log k)$.

Rationale (proposer's reasoning):

The minimal 1-certificate clutter captures the irreducible branching obligations any decision tree must witness; their conflict graph encodes exactly the pairs that cannot coexist on a single root-to-leaf path, so its cycle-richness should constrain protocol/decision-tree branching budgets. The sandpile group is a torsion abelian invariant whose log-size is Kirchhoff's spanning-tree count — a single integer determinant — and a depth- d tree induces at most k^d distinct query histories, plausibly bounding how much such cyclic redundancy can be 'covered'. Chip-firing and sandpile invariants have essentially never been deployed in lifting-style lower bounds, giving an algebraic-combinatorial obstruction distinct from rank, Dilworth-width, Möbius, or asymptotic-dimension routes.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3868f434d3c1924e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 800 trials (200 per $k \in \{3, 4, 5, 6\}$), compute $\Delta(f) = \log_2 \tau(G_f) - Q^{dt}(f) \cdot \log_2(k+1)$. Conjecture is SUPPORTED iff every trial satisfies $\Delta(f) \leq 0$ (i.e. $\max \Delta \leq 1e-9$) and the per- k mean Δ is non-increasing (more negative) as k grows from 3 to 6. FALSIFIED if any single trial yields $\Delta(f) > 1e-9$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - sandpile group certificate complexity Boolean function spanning trees - chip-firing decision tree complexity lifting communication-critical group conflict graph query complexity Raz-McKenzie

Top relevant hits considered: - [http://arxiv.org/abs/0906.2809v2] Sandpile groups and spanning trees of directed line graphs - [http://arxiv.org/abs/2602.01042v2] On Condensation of Block Sensitivity, Certificate Complexity and the AND (and OR) Decision Tree Co - [http://arxiv.org/abs/1904.12209v2] Sandpile monomorphisms and limits - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [http://arxiv.org/abs/0811.2811v1] Parallel Chip-Firing on the Complete Graph: Devil's Staircase and Poincare Rotation Number - [http://arxiv.org/abs/2105.01963v3] One-way communication complexity and non-adaptive decision trees - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/2309.16111v2] The relational complexity of linear groups acting on subspaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_truth_table(k):
        return [random.choice([0, 1]) for _ in range(2**k)]
```

```

def is_minimal_1_certificate(f, C):
    for i in range(len(C)):
        if f[C[:i] + (1 - C[i],) + C[i+1:]] == f[C]:
            return False
    return True

def build_conflict_graph(f, k):
    vertices = [C for C in combinations(range(k), len(C)) if is_minimal_1_certificate(f, C)]
    edges = []
    for C, C_prime in combinations(vertices, 2):
        common_vars = set(i for i in range(k) if C[i] != C_prime[i])
        if any(f[C[:i] + (1 - C[i],) + C[i+1:]] != f[C_prime[:i] + (1 - C_prime[i],) + C_prime[i+1:]]) for i in common_vars):
            edges.append((C, C_prime))
    return vertices, edges

def laplacian_matrix(G):
    n = len(G[0])
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(1 for j in range(n) if G[0][i] == G[0][j])
        L[i][i] = -degree
        for j in range(i + 1, n):
            if (G[0][i], G[0][j]) in G[1]:
                L[i][j] = L[j][i] = 1
    return L

def determinant(M):
    n = len(M)
    det = 0
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in M[1:]]
        sign = (-1) ** i
        if n == 2:
            det += sign * M[0][i] * M[1][1]
        else:
            det += sign * M[0][i] * determinant(submatrix)
    return det

def Q_dt(f, k):
    memo = {}

    def dp(mask):
        if mask in memo:
            return memo[mask]
        if len(mask) == 1:
            return 1
        max_val = -math.inf
        for i in range(len(mask)):
            new_mask = tuple(0 if j == i else mask[j] for j in range(len(mask)))
            max_val = max(max_val, dp(new_mask))
        memo[mask] = max_val
        return max_val

    return dp(tuple(range(k)))

```

```

def minmax_dp(f, k):
    memo = {}

    def dp(mask):
        if mask in memo:
            return memo[mask]
        if len(mask) == 1:
            return 0
        min_val = math.inf
        for i in range(len(mask)):
            new_mask = tuple(0 if j == i else mask[j] for j in range(len(mask)))
            min_val = min(min_val, dp(new_mask))
        memo[mask] = min_val
        return min_val

    return dp(tuple(range(k)))

def log2(x):
    return math.log2(x)

k_values = [3, 4, 5, 6]
results = []
for k in k_values:
    for _ in range(200):
        f = generate_truth_table(k)
        C = list(range(k))
        if not is_minimal_1_certificate(f, C):
            continue
        vertices, edges = build_conflict_graph(f, k)
        L = laplacian_matrix((vertices, edges))
        tau_G_f = determinant(L)
        Q_dt_f = Q_dt(f, k)
        minmax_dp_f = minmax_dp(f, k)
        delta = log2(tau_G_f) - Q_dt_f * log2(k + 1)
        results.append(delta)

mean_delta = sum(results) / len(results)
conjecture_holds = all(delta <= 0 for delta in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "delta",
    "metric_value": mean_delta,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

```

```

mean_delta = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_delta} std=0.0 support_fraction={support_fraction}")
elif any(r["delta"] > 1e-9 for r in results):
    first_failing_seed = next(r["seed"] for r in results if r["delta"] > 1e-9)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f6dc07d.py", line 131, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f6dc07d.py", line 104, in run_trial
    if not is_minimal_1_certificate(f, C):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f6dc07d.py", line 25, in is_minimal_1_certificate
    if f[C[:i] + (1 - C[i],) + C[i+1:]] == f[C]:
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: can only concatenate list (not "tuple") to list

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (list/tuple concatenation bug in `is_minimal_1_certificate`) before any of the 800 trials produced data, so the pre-registered support condition cannot be evaluated. | next: Fix the type bug by representing certificates consistently (e.g., always tuples) so that `C[:i] + (1 - C[i],) + C[i+1:]` type-checks, then rerun the 800-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	123974
2	preregistration	claude_max	opus	0	0	6950
3	novelty	claude_max	opus	0	0	3580
4	novelty	claude_max	opus	0	0	9887
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12473
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14097
7	judge	claude_max	opus	0	0	4807

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 175767 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/ef886a7094f0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ef886a7094f0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ef886a7094f0.tar.gz` (if generated)